B 2.5.1 File Processing Python

B 2.5.1 Construct code to perform file-processing operations

Learn file processing in Java and Python. This resource covers opening, reading, writing, and appending to text files, including error handling and best practices. Hands-on activities, worksheets, and slide decks for both languages provide practical coding experience. Students develop critical thinking and communication skills.

1. Introduction to File Processing

Programs often need to interact with persistent data. While variables hold data temporarily, files store data on disk, allowing programs to save and retrieve information across sessions. This is essential for saving game progress, reading configuration settings, or processing large datasets.

2. Opening Files

The open() function is used to open files in Python. You specify the file name and the mode:

```
# Read mode ('r')
file = open("data.txt", "r")

# Write mode ('w') - Overwrites existing content
file = open("output.txt", "w")

# Append mode ('a') - Adds to the end of the file
file = open("log.txt", "a")
```

3. Reading from Files

Once a file is open in read mode, you can read its content.

```
file = open("data.txt", "r")

# Read the entire file
content = file.read()
print(content)

# Read line by line
for line in file:
    print(line.strip())  # Remove newline characters

# Read all lines into a list
lines = file.readlines()
for line in lines:
    print(line.strip())

file.close()  # Close the file (important!)
```

4. Writing to Files

To write data to a file, open it in write mode ('w') and use the write() method.

```
file = open("output.txt", "w")
file.write("Hello, world!\n")
file.write("This is a new line.\n")
file.close()
```

5. Appending to Files

To add data to an existing file without deleting its contents, open it in append mode ('a').

```
file = open("log.txt", "a")
file.write("A new entry.\n")
file.close()
```

6. The with Statement (Recommended)

The with statement provides a convenient way to close files, even if errors occur automatically. It's the recommended way to work with files in Python.

```
with open("data.txt", "r") as file:
    for line in file:
        print(line.strip())

with open("output.txt", "w") as file:
    file.write("Some text.\n")

with open("log.txt", "a") as file:
    file.write("Another log entry.\n")
```

7. Error Handling

Use try-except blocks to handle potential errors during file operations:

```
try:
    file = open("myfile.txt", "r")
    # ... file operations ...
    file.close()
except FileNotFoundError:
    print("File not found.")
except Exception as e:  # Catch other exceptions
    print("An error occurred: ", e)
```


Report a problem